BL-13: Complete, duplicate-safe attendance submissions

- `submit_attendance` uses `transaction.atomic` for session, records and audit entry.
- `TeachingAssignment.objects.select_for_update()` serializes submissions for the same class and subject on PostgreSQL.
- The actor is reloaded and locked inside the transaction, so a cached active faculty object cannot bypass a later deactivation or role change.
- The unique `(assignment, date, period)` and `(session, student)` constraints remain the final integrity backstop.
- The same statuses submitted again return the existing session, without a second audit entry or student record.
- A different submission for an existing session is rejected. Editing saved attendance belongs to BL-17.
- Failed database writes roll back the session and all records, never leaving partial attendance.
- Original rosters are preserved on identical retries even if a later enrollment was removed.
- Repeated status fields, malformed student IDs, invalid statuses and incomplete rosters are rejected without saving. Service callers also receive validation errors for invalid date, period or status-map types.
- Unexpected database save failures display a clear failure message and roll back the entire submission.

Verification: `python manage.py test apps.attendance.test_submission_safety`; PostgreSQL tests exercise simultaneous identical and conflicting submissions with independent connections. Failure injection covers both student-record inserts and a failure after an audit entry was written. Database constraints are tested directly as the final duplicate guard.

Concurrency test cleanup temporarily permits `TRUNCATE` only in Django's disposable test database, then restores the append-only audit trigger. No production trigger changes are made.